

软件概要设计说明书

观澜视频平台 · 第 4 组 · 2026 夏季软件工程基础实践

可编辑源文件与本 PDF 同时提交；本文由仓库脚本生成，页脚标注页码，最终以 Git 中的 Markdown/Mermaid 源为准。

软件概要设计说明书 V1

[TOC]

1 引言

1.1 编写目的

本文档用于说明观澜视频平台首版系统的总体设计方案，明确系统的分层结构、模块划分、核心业务流程、接口设计、数据结构设计、部署方案和关键设计约束，为后续详细设计说明书、数据库建模、接口实现、前后端联调和测试提供统一的架构基线。

1.2 项目背景

观澜视频平台是软件工程基础课程“在线视频与直播网站”选题的项目交付系统，面向视频观看者、平台用户和平台管理员，提供短视频、中视频、长视频与直播内容的一体化消费和管理能力。系统首版以 Web 端为主要承载形态，产品体验上采用“B 站式社区主站 + 抖音式推荐入口”的组合形式，同时强调审核后台、Agent 审核辅助、真实推流直播和录播回看等课程亮点能力。

1.3 术语与缩略语

- SRS：软件需求规格说明书
- RTMP：直播推流协议
- HLS：直播或录播播放协议
- WebSocket：浏览器与服务端双向实时通信协议
- Agent：用于审核辅助的智能分析模块
- MinIO：对象存储服务
- SRS：Simple Realtime Server，直播推流与播放服务
- FFmpeg：视频转码和媒体处理工具

1.4 参考文档

- [VideoPlayer/docs/项目计划书-V1.md](#)
- [VideoPlayer/docs/开发手册-V1.md](#)
- [VideoPlayer/docs/软件需求规格说明书-V1.md](#)
- [VideoPlayer/docs/数据库设计-V1.md](#)
- [VideoPlayer/docs/API文档-V1.md](#)

2 系统需求概述

2.1 需求规定

系统首版必须覆盖以下核心需求：

- CR-01 内容消费与分发
- CR-02 内容发布与审核

- CR-03 社区互动
- CR-04 关注订阅
- CR-05 直播互动
- CR-06 用户中心
- CR-07 平台治理与 Agent

系统必须支持游客、用户、管理员三类角色，并保证角色边界清晰、功能链路完整、交付文档与实现一致。

2.2 业务目标

- 提供统一的视频与直播消费入口
- 支持用户从投稿到审核发布、从开播到录播回看的完整链路
- 为普通用户提供评论、二级回复、视频弹幕、直播聊天与送礼等互动能力
- 为平台管理员提供审核、举报处理、处罚与运营概览能力
- 为管理员提供 Agent 风险提示，提升审核效率并体现平台治理能力

2.3 运行环境

2.3.1 客户端环境

- 浏览器：Chrome、Edge、Firefox 最新稳定版
- 分辨率：优先适配 1440px 及以上 PC 布局，兼容常规笔记本分辨率

2.3.2 服务端环境

- Node.js 20+
- MySQL 8
- Redis 6+
- MinIO
- FFmpeg
- SRS
- Nginx
- Docker / Docker Compose

2.4 设计约束

2.4.1 业务约束

- 审核策略固定为先审后发
- 视频弹幕与直播弹幕必须分离实现
- 游客只允许浏览和播放，不允许互动或创作
- 礼物币只做模拟资产，不接真实支付
- 首版推荐采用规则版，不引入训练型推荐系统

2.4.2 工程约束

- 后端采用单体模块化设计，不采用微服务拆分

- 前后端统一使用 TypeScript，降低协作成本
- 所有核心状态必须有明确状态机
- 所有外部依赖需支持本地或演示环境部署

2.4.3 文档约束

- 文档中的模块名称、状态名、接口命名和表名必须一致
- 所有图片和图表后续统一使用仓库相对路径保存
- 设计说明必须能映射到课程答辩中的核心需求覆盖表

2.5 功能需求概要

系统主要由以下八个功能模块组成：

1. 用户与权限模块
2. 视频内容模块
3. 推荐与搜索模块
4. 互动模块
5. 直播模块
6. 用户中心模块
7. 审核与运营模块
8. 礼物币与 Agent 模块

2.6 非功能需求

- 可用性：主要用户流程必须连续可操作
- 性能：在课程演示环境的小并发场景下稳定运行
- 安全性：实现鉴权、权限校验、上传校验、审计日志
- 可维护性：模块边界清晰，接口统一，状态机明确
- 可扩展性：后续能扩展推荐策略、直播玩法和 Agent 能力

3 系统总体架构设计

3.1 架构设计原则

- 前后端分离
- 单体模块化后端
- 媒体能力独立外置
- 审核与治理贯穿内容全生命周期
- 实时能力和离线能力分层处理

3.2 总体架构

系统总体架构由五层组成：

1. 表现层：Web 主站、用户中心、管理后台
2. 接入层：Nginx、鉴权拦截、统一网关入口

3. 业务层：NestJS 模块化业务服务
4. 数据与媒体层：MySQL、Redis、MinIO、FFmpeg、SRS
5. 智能辅助层：Agent 审核分析服务

3.3 分层结构说明

3.3.1 表现层

表现层由 Vue 3 单页应用组成，按业务域拆分为：

- 主站消费页面
- 用户中心页面
- 管理后台页面

表现层负责：

- 页面渲染
- 交互控制
- 路由切换
- 状态管理
- WebSocket 消息订阅

3.3.2 接入层

接入层主要由 Nginx 和后端统一鉴权逻辑组成，负责：

- 静态资源访问
- API 转发
- 鉴权前置控制
- 跨域与基础安全控制
- 直播播放地址转发

3.3.3 业务层

业务层由 NestJS 单体模块化服务构成，负责：

- 用户认证与权限校验
- 内容发布、审核和播放
- 互动行为处理
- 关注订阅
- 直播管理与录播管理
- 运营治理逻辑
- 礼物币和通知逻辑
- Agent 风险分析结果接入

3.3.4 数据与媒体层

- MySQL：存储结构化业务数据

- Redis：缓存热点数据、临时状态和消息辅助数据
- MinIO：存储视频、封面、录播和媒体产物
- FFmpeg：处理转码、抽帧、录播加工
- SRS：处理直播推流、转流与播放

3.3.5 智能辅助层

Agent 服务用于审核辅助，输入为标题、简介、评论、弹幕等文本信息，输出为风险等级、命中规则、建议动作和摘要说明。该层不直接执行处罚或删除动作，只将结果提供给审核模块使用。系统管理员入口通过管理密钥校验后才允许显示和使用。

4 模块设计

4.1 模块划分总览

模块	职责范围	关键对象
用户与权限模块	登录、注册、角色权限、资料维护	user, user_profile
视频内容模块	投稿、媒体处理、详情播放、状态流转	video, video_asset, video_review
推荐与搜索模块	首页推荐、分区、热榜、搜索	category, search_hotword
互动模块	点赞、收藏、评论、视频弹幕、通知	comment, video_danmaku, favorite, notification
直播模块	直播间、场次、聊天、直播弹幕、录播	live_room, live_session, live_chat_message, live_danmaku, live_recording
用户中心模块	数据概览、作品管理、直播管理、违规提醒	user studio 聚合对象
审核与运营模块	视频审核、举报处理、处罚、运营看板	report_record, audit_log, video_review
礼物币与 Agent 模块	每日领取、送礼、风险提示	gift_coin_wallet, gift_order, agent_review_result

4.2 用户与权限模块

4.2.1 职责

- 处理注册、登录和退出
- 维护用户基础资料
- 在接口层和页面层识别不同角色
- 为其他模块提供当前用户上下文

4.2.2 设计说明

- 游客态不落库，仅在前端表现层存在
- 登录态通过 Bearer Token 鉴权
- 用户角色首版采用用户表内角色字段
- 权限控制以路由守卫和后端装饰器双重实现

4.3 视频内容模块

4.3.1 职责

- 接收视频上传
- 保存草稿和视频元数据

- 触发媒体处理任务
- 管理投稿状态流转
- 提供视频详情页所需内容

4.3.2 设计说明

- 上传后先记录视频草稿，再关联媒体资源
- 媒体处理和审核提交是两个独立步骤
- 视频状态机采用：DRAFT -> PROCESSING -> PENDING_REVIEW -> PUBLISHED/REJECTED/OFFLINE
- 短视频、中视频、长视频共享同一实体模型，通过 video_type 区分

4.4 推荐与搜索模块

4.4.1 职责

- 首页推荐流生成
- 分区内容列表输出
- 热门榜单生成
- 全局搜索结果聚合

4.4.2 设计说明

- 首版推荐采用规则打分，不做召回/排序多阶段拆分
- 推荐只面向审核通过且公开的视频
- 搜索结果统一入口，结果页按视频、直播、用户分 Tab 输出
- 热词统计可落到数据库，也可由 Redis 辅助统计后定期回写

4.5 互动模块

4.5.1 职责

- 点赞、收藏
- 评论与二级回复
- 视频弹幕
- 通知生成
- 举报入口

4.5.2 设计说明

- 评论采用树形结构，一级评论和二级回复通过 parent_id 与 root_id 组织
- 视频弹幕按时间轴偏移存储，播放时分段拉取
- 点赞与收藏采用关系记录 + 计数字段冗余方案
- 互动产生事件后可异步触发通知生成

4.6 直播模块

4.6.1 职责

- 直播间创建与配置
- 获取推流信息
- 开始和结束直播
- 直播聊天和直播弹幕
- 录播回看

4.6.2 设计说明

- 直播间是长期对象，直播场次是一次开播行为对应的短周期对象
- 推流使用 RTMP，播放使用 HLS 或 HTTP-FLV
- 聊天和直播弹幕通过 WebSocket 广播
- 直播结束后由录播处理任务生成 `live_recording`
- 直播状态区分直播间状态和直播场次状态，避免一个对象承担双重职责

4.7 用户中心模块

4.7.1 职责

- 聚合作品数据、直播数据和粉丝数据
- 提供作品管理和直播管理界面
- 展示粉丝趋势与违规提醒

4.7.2 设计说明

- 用户中心更多是聚合视图层，不独立维护大量核心实体
- 数据概览优先读取统计字段或聚合查询结果
- 违规提醒来源于审核驳回记录、处罚记录或举报处理结果

4.8 审核与运营模块

4.8.1 职责

- 视频审核
- 评论和弹幕审核
- 举报处理
- 用户处罚
- 运营概览展示

4.8.2 设计说明

- 审核流与内容流严格分离
- 管理员动作需要写入审计日志
- 审核后台优先显示高风险内容与待处理内容

- 举报对象通过 `target_type + target_id` 抽象，不为每种类型单独建一套处理表

4.9 礼物币与 Agent 模块

4.9.1 礼物币职责

- 每日登录领取礼物币
- 钱包余额维护
- 直播送礼记录

4.9.2 Agent 职责

- 提供风险识别结果
- 输出风险等级和建议动作
- 将审核辅助结果持久化

4.9.3 设计说明

- 礼物币不与真实支付打通，因此只维护余额和流水，不做提现和收益结算
- Agent 结果必须可追踪、可查看、可驳回
- Agent 处理失败不影响人工审核链路

5 核心业务流程设计

5.1 视频投稿与审核流程

1. 用户上传原始视频文件
2. 系统创建草稿并保存元数据
3. 媒体处理模块进行转码、抽帧、时长解析
4. 用户提交审核
5. Agent 预分析文本风险
6. 管理员审核通过或驳回
7. 审核通过内容进入推荐、搜索和公开视频范围

5.2 视频消费与互动流程

1. 用户进入首页推荐流或分区页
2. 前端请求推荐接口或搜索接口
3. 用户进入视频详情页
4. 系统加载播放信息、评论、相关推荐和弹幕
5. 用户进行点赞、收藏、评论、二级回复、发送视频弹幕等行为
6. 系统更新统计字段并生成通知

5.3 直播流程

1. 用户创建直播间
2. 系统返回推流地址与推流密钥

3. 用户通过推流工具发起直播
4. SRS 接收推流并生成播放地址
5. 用户进入直播间观看
6. 用户发送直播聊天、直播弹幕和礼物
7. 主播结束直播
8. 系统生成录播记录供回看

5.4 举报与治理流程

1. 用户对视频、评论或弹幕发起举报
2. 系统保存举报记录
3. Agent 对相关文本生成风险建议
4. 管理员查看审核后台中的待处理对象
5. 管理员执行保留、删除、禁言、封禁等处理
6. 系统记录审计日志和处理结果

6 状态机设计

6.1 视频状态机

- **DRAFT** : 草稿未提交
- **PROCESSING** : 媒体处理中
- **PENDING_REVIEW** : 待审核
- **REJECTED** : 审核驳回
- **PUBLISHED** : 已发布
- **OFFLINE** : 已下架

状态流转原则：

- 草稿可编辑
- 处理中不可提交审核
- 驳回后可编辑并重新提交
- 只有已发布状态才能进入公开分发

6.2 直播状态机

6.2.1 直播间状态

- **CREATED**
- **READY**
- **LIVING**
- **ENDED**
- **BANNED**

6.2.2 直播场次状态

- READY
- LIVING
- ENDED
- ABNORMAL

6.3 举报状态机

- PENDING
- PROCESSED
- REJECTED

7 接口设计概要

7.1 用户接口设计

7.1.1 主站界面

主站界面包含首页、分区页、搜索结果页、视频详情页、直播间页、用户主页等页面，强调内容消费与互动。首页推荐页采用推荐英雄区 + 视频卡片网格布局；搜索结果页按视频和用户分 Tab 展示；视频详情页在主播放器外补充相关推荐区。

7.1.2 用户中心界面

用户中心界面包含数据概览、作品管理、直播管理、粉丝趋势和违规提醒，强调管理效率和状态可见性。

7.1.3 管理后台界面

管理后台包含审核工作台、文本审核、举报处理、用户处罚与运营看板，强调审核效率和治理视角。

7.2 外部软件接口设计

外部组件	接口作用
MySQL	结构化数据存储
Redis	缓存、热点数据、辅助实时能力
MinIO	视频、封面、录播对象存储
FFmpeg	媒体处理
SRS	直播推流与播放分发
Agent 服务	风险识别和审核建议

7.3 内部模块接口设计

- 认证模块对其他模块提供当前用户上下文
- 视频模块向推荐模块输出公开视频数据源
- 审核模块调用 Agent 模块获取风险建议
- 互动模块调用通知模块生成消息
- 直播模块调用礼物币模块完成送礼扣减和记录

7.4 实时接口设计

- 直播聊天采用 WebSocket 广播
- 直播弹幕采用 WebSocket 广播
- 视频弹幕采用 HTTP 拉取 + 时间轴显示
- 审核结果和系统消息采用通知推送或轮询获取

8 数据结构设计概要

8.1 核心实体

系统核心实体包括：

- 用户与资料
- 视频与媒体资源
- 视频审核
- 评论与视频弹幕
- 关注关系
- 直播间与直播场次
- 直播聊天与直播弹幕
- 录播资源
- 礼物币钱包和流水
- 举报记录
- 审计日志
- Agent 审核结果
- 通知

8.2 数据关系说明

- 一个用户可以拥有多个视频和多个直播间
- 一个直播间可以产生多场直播记录
- 一个视频可以有多条评论和多条视频弹幕
- 一个直播场次可以有多条聊天消息、多条直播弹幕和一条录播记录
- 一个审核对象可以关联一条或多条 Agent 风险结果

8.3 冗余字段设计

为提高首版查询效率，系统对以下数据采用计数冗余：

- 视频播放量、点赞数、收藏数、评论数
- 评论点赞数、回复数
- 钱包余额、累计领取和累计消费
- 直播场次峰值在线人数

9 部署设计

9.1 部署拓扑

首版部署采用 Docker Compose 统一管理，包含以下容器：

- **frontend**：前端静态资源
- **backend**：NestJS 后端
- **mysql**：关系型数据库
- **redis**：缓存
- **minio**：对象存储
- **srs**：直播服务
- **nginx**：统一入口

9.2 请求链路

9.2.1 常规页面访问

浏览器 -> Nginx -> 前端静态资源 / 后端 API -> MySQL / Redis / MinIO

9.2.2 直播访问链路

推流工具 -> SRS -> 播放地址 -> 浏览器播放器

9.2.3 媒体处理链路

用户上传视频 -> 后端记录任务 -> FFmpeg 处理 -> MinIO 存储结果 -> 后端更新状态

9.3 运行控制

- 后端启动时校验数据库与缓存连接
- 媒体处理服务需要检测 FFmpeg 可用性
- 直播服务需要校验 SRS 状态和推流配置
- 演示环境需要预置测试账号、测试视频和测试直播间

10 安全与质量设计

10.1 安全设计

- 使用密码哈希存储密码
- 所有用户和管理员接口进行权限控制
- 上传文件做类型、大小和后缀校验
- 关键操作写入审计日志
- 举报处理和处罚动作必须可追溯

10.2 治理设计

- 审核流和公开流分离
- 评论和弹幕支持举报和审核

- Agent 结果仅辅助人工决策
- 用户处罚支持禁言、封禁等最小治理动作

10.3 性能设计

- 首页推荐、视频详情和搜索结果使用分页
- 热点统计和热词可用 Redis 辅助
- 视频弹幕按时间窗口查询
- 直播聊天与直播弹幕采用轻量广播机制

10.4 可维护性设计

- 前后端统一 TypeScript
- 模块命名与文档命名统一
- 状态名统一维护为常量或枚举
- 接口输出统一格式化

11 设计特点与权衡

11.1 选择单体模块化而非微服务

理由：

- 课程项目更重视可落地性和文档一致性
- 团队规模有限，微服务会增加部署和联调复杂度
- 直播和媒体处理的复杂度已经足够高，后端无需再人为增加分布式成本

11.2 选择规则版推荐而非智能推荐

理由：

- 规则版更适合文档化说明和答辩展示
- 数据量有限，不适合做训练型推荐
- 规则版足以支撑首版首页推荐和分区内容排序

11.3 选择 Agent 辅助而非自动审核

理由：

- 可降低误判带来的错误处理风险
- 更符合课程项目中的“可解释、可展示”要求
- 便于管理员在后台中直观展示 AI 风险提示能力

12 当前待补充项

- 系统架构图、部署图、状态机图和主要时序图尚未绘制成正式图片
- 录播处理是否完全依赖 SRS 产物或需要 FFmpeg 二次处理仍需在实现时确认
- 用户中心统计是否通过实时聚合还是每日离线汇总，需在实现阶段进一步决定

13 结论

本概要设计说明书将系统首版架构明确为“前后端分离 + 单体模块化后端 + 外置媒体服务 + Agent 审核辅助”的设计路线。该方案在满足课程规模、展示效果、文档完整性和团队实现能力之间取得平衡，可直接作为后续详细设计说明书和工程落地的上游依据。